



Tai Chi

Isle v2 Security Review Report

TaiChi Audit Group

March 24, 2026

Contents

1	About TaiChi Audit Group	4
2	Risk Classification	4
2.1	Impact	4
2.2	Likelihood	4
3	Security Assessment Summary	5
3.1	Project Summary	6
3.2	Codebase high-level walkthrough	6
3.3	Scope	7
3.4	Findings	7
3.5	High Risk Findings	7
3.5.1	[H-1] Factory Deploys Markets Without Registering Upgrade Facet Selectors, Making Diamond Upgrades Unreachable	7
3.5.2	[H-2] Protocol Fee Shares from topUpFromService Are Minted But Never Credited to Fee Recipient	11
3.5.3	[H-3] Collateral-input liquidation can leave liquidatable debt unresolved indefinitely	14
3.6	Medium Risk Findings	19
3.6.1	[M-1] In theIsleV2Service.sol, the makeRepay() under RepayMode.General mode allows shares > 0 && assets > 0, which can lead to inaccurate previewRepay().	19
3.6.2	[M-2] Grace Period Debt Overwrite in _accrueCurrentPeriod Case B if gracePeriodDuration > periodLength	23
3.6.3	[M-3] Accrual Writeback Overwrites Fee Shares When Borrower Equals Fee Recipient	27

3.6.4	[M-4] Fee Rates Used at executePayment Differ from Creation Time, Enabling In-Flight Over-Charging .	28
3.6.5	[M-5] Missing Slippage Protection in Supply and Withdraw Operations	31
3.6.6	[M-6] Service previews do not faithfully match execution and can underfund after accrual	34
3.6.7	[M-7] topUpFromService mints protocol fee shares against stale supply state	37
3.6.8	[M-8] Bad debt skipped when collateral exhausted later	39
3.6.9	[M-9] Fee configuration changes retroactively redirect pending accrued fees	44
3.6.10	[M-10] Deleverage close factor is not capped at maxCF above main threshold	47
3.7	Low Risk Findings	49
3.7.1	[L-1] Missing Event Emission on state update functions	49
3.7.2	[L-2] Salt Collision in _calcSalt Prevents Deployment of Reversed Collateral/Loan Markets	50
3.7.3	[L-3] Desynchronization Between defaultAdmin and DEFAULT_ADMIN_ROLE & default-admin handover does not revoke legacy privileged roles	51
3.7.4	[L-4] Diamond Proxy receive() Permanently Locks Native ETH Due to Missing Withdrawal Mechanism . .	53
3.7.5	[L-5] IRM Change Causes Inconsistent Interest Rates Between Global Accrual and Grace/Current Period Debt Conversion	54
3.7.6	[L-6] Inconsistent Role Requirements for setFeeBps Between Market and Service	57
3.7.7	[L-7] Repay Signatures Do Not Bind to Mode: Operator Can Redirect Signature	59
3.7.8	[L-8] Liquidate and Deleverage Lack Slippage Protection for Liquidators	61

3.8	Informational Findings	65
3.8.1	[I-1] Missing ‘__AccessControl_init()’ Call in IsleV2Factory Initializer	65
3.8.2	[I-2] Period Selection Uses “Largest Start” Instead of “Longest Remaining Time”	66
3.8.3	[I-3] Payment Cancellation Repay-to-Pool Ignores Share-Based Debt	68
3.8.4	[I-4] Interest Continues to Accumulate During Protocol Pause	71

1 About TaiChi Audit Group

TaiChi Audit Group is a team composed of DeFiHackLabs community white hat hats with extensive experience in competitive audit platforms/CTF. We are dedicated to enhancing project security coverage and providing comprehensive protection for funds.

2 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	Medium	Low
Likelihood: Medium	Medium	Low	Informational
Likelihood: Low	Low	Informational	Informational

2.1 Impact

- High: leads to a loss of assets in the protocol, or significant harm to a majority of users.
- Medium: function or availability of the protocol could be impacted or losses to only a subset of users.
- Low: State handling, function incorrect as to spec, issues with clarity, losses will be annoying but bearable.

2.2 Likelihood

- High: almost certain to happen, easy to perform, or not easy but highly incentivized.

- Medium: only conditionally possible or incentivized, but still relatively likely.
- Low: requires stars to align, or little-to-no incentive.

3 Security Assessment Summary

Taichi Audit performed a security review of the Isle v2 protocol from February 27 to March 7, covering the lending-market factory, the diamond-based market implementation, liquidation and deleverage flows, interest accrual, and the periphery payment service.

The review identified 25 findings in total: 3 High, 10 Medium, 8 Low, and 4 Informational. The most significant issues affected upgradeability of factory-deployed markets, fee-accounting correctness, and liquidation behavior under certain collateral-input paths. Medium-severity findings were concentrated around accrual ordering, fee configuration, payment execution semantics, and service preview mismatches. Low and Informational findings mainly covered role-model consistency, operational safety, and best-practice gaps.

Based on the issue-thread responses reflected in this report, 21 findings were fixed and 4 were acknowledged. All 3 High and 10 Medium findings were fixed.

The white hats are:

- [@icebear](#)
- [@ret2basic](#)
- [@jesjupyter](#)

Disclaimer: This review does not guarantee against a hack. It is a snapshot in a time of commit (commit hash) according to the specific commit. Any modifications to the code will require a new security review.

3.1 Project Summary

3.2 Codebase high-level walkthrough

Isle v2 is a modular lending protocol built around a factory that deploys isolated lending markets as diamond proxies. Each market is composed of facets that split responsibility across authorization, configuration, market-wide accounting, supplier actions, borrower actions, liquidation and deleverage, initialization, and view helpers.

At deployment time, the factory assembles the facet set and initializes a new market with the selected collateral token, loan token, oracle, interest-rate model, and period oracle. Once deployed, the market tracks supplier shares, borrower debt buckets, collateral balances, global borrow and supply totals, and role-gated configuration such as thresholds, fee settings, and oracle addresses.

Interest accrual is lazy and time-based. Global market accrual updates aggregate borrow and supply state, while borrower-specific accrual converts debt across current-period, grace-period, and overdue share-based buckets. Liquidation and deleverage flows depend on live oracle pricing, risk thresholds, close-factor parameters, and liquidation incentives to determine how much debt can be repaid and how much collateral can be seized.

The periphery `IsleV2Service` contract provides signed meta-transaction style entrypoints for supply, repay, and payment execution. It coordinates token movement on behalf of users, interacts with markets through previews and state-changing calls, and routes payment fees between suppliers and the protocol. The `PeriodOracle` supplies repayment scheduling data, while the oracle and IRM components drive collateral valuation and interest-rate behavior across all markets.

3.3 Scope

Summary

Project Name	Isle v2
Repository	https://github.com/isle-labs/isle-v2-contracts
Commit hash	165ad90a3e1a38cb8c27ea2a2d9095da513f3dcf
File in Scope	every file under src/

Issues Found

Severity	Count	Fixed	Acknowledged
Crit	0	0	0
High	3	3	0
Medium	10	10	0
Low	8	6	2
Informational	4	2	2
Total	25	21	4

3.4 Findings

3.5 High Risk Findings

3.5.1 [H-1] Factory Deploys Markets Without Registering Upgrade Facet Selectors, Making Diamond Upgrades Unreachable

Summary

`IsleV2Factory._buildFacetInits()` only registers 7 facets (`authorization`, `market`, `supplier`, `borrower`, `liquidator`, `configuration`, `viewer`) and omits `initializerFacet`. The `InitializerFacet` is passed to the Diamond constructor only as the one-time `initContract` for initialization, not as a facet whose selectors are installed. As a result, `setFacet`, `updateFacet`, `removeFacet`, and `getFacet` are never registered in the Diamond's `selectorToFacet` mapping, so upgrade operations cannot be called on factory-deployed markets.

Technical Details

Factory `_buildFacetInits` omits `InitializerFacet` from `FacetInit` array

```
1      /// @dev Builds the FacetInit array for Diamond
      deployment.
2      function _buildFacetInits() internal view returns (
          FacetInit[] memory inits) {
3          FacetAddresses storage f = __getFactoryStorage().
              facets;
4
5          inits = new FacetInit[](7);
6          inits[0] = _buildFacetInit(f.authorizationFacet);
7          inits[1] = _buildFacetInit(f.marketFacet);
8          inits[2] = _buildFacetInit(f.supplierFacet);
9          inits[3] = _buildFacetInit(f.borrowerFacet);
10         inits[4] = _buildFacetInit(f.liquidatorFacet);
11         inits[5] = _buildFacetInit(f.configurationFacet);
12         inits[6] = _buildFacetInit(f.viewerFacet);
13     }
```

`initializerFacet` is stored in `FacetAddresses` but never added to `inits`. For `initializerFacet`, it's only used only as `init` target, not as a facet.

```
1      // Deploy Diamond with deterministic salt
2      bytes32 salt = _calcSalt(collateralToken,
3                               loanToken);
4      IsleV2Diamond diamond = new IsleV2Diamond{ salt:
5                               salt }(facetInits, $.facets.initializerFacet,
6                               initData);
```

`$.facets.initializerFacet` is passed as the second argument (initContract), which is used only for a single `delegatecall(initData)` during construction. It is not used to populate the selector routing table.

Since Diamond routing depends on registered selectors, and only selectors from facets are registered, `initContract` is never added to `selectorToFacet`, so its selectors are never routable.

```
1      constructor(FacetInit[] memory facets, address
2                  initContract, bytes memory initData) {
3          DiamondStorage storage ds = __getDiamondStorage();
4
5          // Register all facets
6          unchecked {
7              for (uint256 i = 0; i < facets.length; i++) {
8                  // ... registers facets[i].selectors into
9                  // ds.selectorToFacet
10             }
11         }
12
13         // Call initialization contract if provided
14         if (initContract != address(0)) {
15             (bool success,) = initContract.delegatecall(
16                 initData);
17             // ...
18         }
19     }
```

```
1     function _implementation() internal view override
      returns (address facet) {
2         facet = __getDiamondStorage().selectorToFacet[msg.sig];
3         if (facet == address(0)) {
4             revert ErrorsLib.
              IsleV2Diamond_FunctionNotFound(msg.sig);
5         }
6     }
```

We can take a look at the `facetSelectors` of `inititalizeFacet`:

```
1     /// @inheritdoc IFacet
2     function facetSelectors() external pure returns (
      bytes4[] memory selectors) {
3         selectors = new bytes4[](5);
4         selectors[0] = this.initializer1.selector;
5         selectors[1] = this.setFacet.selector;
6         selectors[2] = this.updateFacet.selector;
7         selectors[3] = this.removeFacet.selector;
8         selectors[4] = this.getFacet.selector;
9     }
```

`setFacet`, `updateFacet`, `removeFacet`, and `getFacet` are defined on `InitializerFacet` but never registered in the Diamond when deployed via the factory.

Impact

- Upgrade functions (`setFacet`, `updateFacet`, `removeFacet`) cannot be called on factory-deployed markets. `getFacet` is also unreachable.
- Markets cannot be upgraded in-place; critical bugs or configuration changes require full migration to new markets.

Recommendation

Include `initializerFacet` in `_buildFacetInits()` so its selectors are registered in the Diamond:

Developer Response

Fixed in [commit 6c734a7](#).

3.5.2 [H-2] Protocol Fee Shares from `topUpFromService` Are Minted But Never Credited to Fee Recipient

Summary

When `topUpFromService(supplierFee, protocolFee)` is called by the payment service, the protocol fee assets are added to the market and `protocolFeeShares` are minted and added to `totalSupplyShares`. However, these shares are **never credited to any user position**—unlike interest-based protocol fees, which are correctly assigned to `config.recipient`. As a result, the protocol fee value is permanently trapped in the pool and cannot be withdrawn by the fee recipient.

Technical Details

Per [docs/payment-service.md](#):

```
Fees are distributed via market.topUpFromService(supplierFee, protocolFee): - supplierFee is added to totalSupplyAssets (accrued to suppliers) - protocolFee is transferred to the market's fee recipient
```

The market has a `config.recipient` that receives protocol fees from interest accrual. The same recipient is expected to receive protocol fees from the payment service.

In `MarketShared.sol`, `_collectFee` correctly credits fee shares to the recipient:

```
1     function _collectFee(  
2         uint256 interest,  
3         MarketData memory $data,  
4         Position memory $recipient  
5     )  
6     private  
7     view  
8     returns (uint256 feeShares)  
9     {  
10        MarketStorage storage $market = __getMarketStorage  
11            ();  
12        if (interest != 0 && $market.config.feeBps != 0 &&  
13            $market.config.recipient != address(0)) {  
14            uint256 feeAssets = interest.mulDiv($market.  
15                config.feeBps, MAX_BPS);  
16            feeShares = feeAssets.toShareDown($data.  
17                totalSupplyAssets - feeAssets, $data.  
18                totalSupplyShares);  
19            $recipient.supplyShares += feeShares;  
20            $data.totalSupplyShares += feeShares;  
21        }  
22    }
```

Here, `$recipient.supplyShares` is updated so the fee recipient can later withdraw the fee value.

In `MarketFacet.sol`, protocol fee shares are minted but never assigned:

```
1      function topUpFromService(uint256 supplierFee, uint256
      protocolFee) external onlyRole(SERVICE_ROLE) {
2          MarketStorage storage $market = __getMarketStorage
            ();
3
4          // Transfer funds from the caller (Service
            contract)
5          uint256 totalFee = supplierFee + protocolFee;
6          if (totalFee > 0) {
7              IERC20($market.config.loanToken).
                safeTransferFrom(msg.sender, address(this),
                    totalFee);
8          }
9
10         // Top up fee for supplier
11         $market.data.totalSupplyAssets += supplierFee;
12
13         // Top up fee for protocol
14         uint256 protocolFeeShares =
15             protocolFee.toShareDown($market.data.
                totalSupplyAssets, $market.data.
                    totalSupplyShares);
16         $market.data.totalSupplyAssets += protocolFee;
17         $market.data.totalSupplyShares +=
            protocolFeeShares;
18
19         emit EventsLib.TopUpFromService(address(this),
            supplierFee, protocolFee);
20     }
```

- `protocolFee` is added to `totalSupplyAssets`
- `protocolFeeShares` is added to `totalSupplyShares`
- **No write to `userPosition[recipient].supplyShares`**—the shares are unowned

Impact

1. **Protocol revenue loss:** All protocol fees from the payment service (`IsleV2Service` payments with `protocolFeeBps > 0`) are effectively locked in the pool and cannot be claimed by the fee recipient.

2. **Documentation mismatch:** The docs state that `protocolFee` is transferred to the fee recipient, but the implementation does not credit the recipient.
3. **Inconsistency with interest fees:** Interest-based protocol fees are correctly credited to `config.recipient`, while payment-service protocol fees are not, leading to inconsistent and incorrect behavior.

Recommendation

Credit `protocolFeeShares` to the fee recipient when `config.recipient != address(0)`, mirroring the interest fee logic in `_collectFee`

Developer Response

Fixed in [commit 5017667](#).

3.5.3 [H-3] Collateral-input liquidation can leave liquidatable debt unresolved indefinitely

Summary

When liquidators specify `collaterals` instead of `shares` or `assets`, the liquidation path caps seized collateral at debt-equivalent collateral without applying the liquidation incentive factor. The function then derives repayable debt by dividing the seized collateral by `lif`, so the liquidation under-repays and cannot fully close debt even when the borrower has enough collateral to support a full close under the protocol's own incentive model.

It breaks a core safety path: the protocol can execute a liquidation on a liquidatable position, consume collateral, and still leave the position partially unresolved purely because the caller chose the collateral-input branch. Since the protocol has no obligation mechanism that forces any actor to come back and finish cleanup, the residual unhealthy position can persist indefinitely.

Technical Details

In the collateral-input branch of `__liquidateForShares()`, the cap is:

```
1      } else {
2          // Case B: Input is collaterals → calculate
           how much debt is repaid
3          // Seize minimum of: requested, available, and
           debt-equivalent collateral
4          seizeCollaterals = Math.min(
5              Math.min(accountingData.inputCollaterals,
6                  $borrower.collaterals),
7              _convertToCollateral(
8                  $borrower.borrowShares.toAssetDown(
9                      $data.totalBorrowAssets, $data.
10                         totalBorrowShares)
11              )
12          );
13          if (seizeCollaterals > 0) {
14              // Reverse calculate debt repaid from
15                 seized collateral
16              repayAssets = _convertToLoan(
17                  seizeCollaterals.wDivUp(lif));
18              repayShares = repayAssets.toShareUp($data.
19                  totalBorrowAssets, $data.
20                  totalBorrowShares);
21              accountingData.inputCollaterals -=
22                  seizeCollaterals;
23          }
24      }
```

The debt-equivalent collateral cap does not include `lif`:

- the cap allows seizing at most `collateralEquivalent(debt)`
- but repayable debt is later computed as `loanEquivalent(seizedCollateral / lif)`

So the maximum repayable debt is reduced by roughly $1 / lif$.

If the borrower has enough collateral to cover `debt * lif`, the caller would expect a collateral-specified liquidation to be able to fully close the debt. Instead, the branch stops early and leaves a residual position.

```
1      if (seizeCollaterals > 0) {
2          // Reverse calculate debt repaid from
           seized collateral
3          repayAssets = _convertToLoan(
           seizeCollaterals.wDivUp(lif));
4          repayShares = repayAssets.toShareUp($data.
           totalBorrowAssets, $data.
           totalBorrowShares);
5          accountingData.inputCollaterals -=
           seizeCollaterals;
6      }
```

The residual debt does not automatically become bad debt in this scenario:

```
1      // Step 5: Update borrower position
2      $borrower.borrowShares -= repayShares;
3      $borrower.collaterals -= seizeCollaterals;
4
5      // Step 6: Handle bad debt if borrower has no
           collateral left
6      if ($borrower.collaterals == 0) {
7          // All remaining debt becomes bad debt (
           socialized loss)
8          badDebtShares = $borrower.borrowShares;
9          badDebtAssets = Math.min(
10             $data.totalBorrowAssets, badDebtShares.
               toAssetUp($data.totalBorrowAssets,
               $data.totalBorrowShares)
11         );
12
13         accountingData.accumulatedBadDebtShares +=
           badDebtShares;
14         accountingData.accumulatedBadDebtAssets +=
           badDebtAssets;
15
16         // Socialize bad debt: reduce both borrow and
           supply
17         $data.totalBorrowAssets -= badDebtAssets;
18         $data.totalSupplyAssets -= badDebtAssets;
19         $data.totalBorrowShares -= badDebtShares;
20
21         $borrower.borrowShares = 0;
22     }
```

- Assume debt = 100 and `lif` = 1.1, and the borrower has collateral worth 110.
- In collateral-input mode, the cap allows at most collateral equivalent to 100, not 110.
- Repayment is then computed as $100 / 1.1 = 90.9$.
- Residual debt of about 9.1 remains even though the borrower had enough collateral to support a full close under the same incentive model.
- Residual collateral of about 10 also remains, so the `if ($borrower.collaterals == 0)` bad-debt branch is not entered in this call.

That means the protocol can be left in an unresolved state where:

- the borrower was liquidatable,
- a liquidation was executed,
- collateral was consumed,
- but the debt was not fully resolved,
- and no bad debt was recognized.

The code assumes that eventual cleanup will happen via some future liquidation attempt, but nothing in the protocol guarantees that.

This pattern is not isolated to `__liquidateForShares()`. The same collateral-input shape exists in the grace-period and current-period branches:

- cap seized collateral at debt-equivalent collateral
- then derive repayable debt by dividing by `lif`

So the issue affects all three debt buckets, not just overdue share debt.

Impact

High.

Liquidation is a core solvency-preserving path. This bug allows a liquidator to choose a valid liquidation mode that is structurally incapable of fully re-

solving debt that should be closable under the protocol's own pricing and incentive model.

The consequence is not merely that liquidation is inefficient. The protocol can be left depending on voluntary future cleanup for a position that was already eligible for forced resolution.

The realistic failure mode is:

1. a position becomes liquidatable,
2. a liquidator executes the collateral-input branch,
3. the protocol consumes collateral but under-repays debt,
4. bad debt is not recognized because collateral is not yet zero,
5. no actor is obligated to come back and finish the job.

That leaves the market with a residual unhealthy position whose cleanup is socially optional.

This matters because:

- permissioned-liquidation markets may have only a small set of authorized liquidators,
- residual dust or low-profit remnants may be economically unattractive to resolve,
- market solvency depends on liquidation being able to reliably finish positions when called.

In effect, a borrower position that should be forcibly resolvable can remain partially open indefinitely, with loss recognition deferred until some later actor voluntarily cleans it up. That is a meaningful failure of the liquidation mechanism, not just poor user experience.

Recommendation

When `collaterals` is the input, the collateral cap for a full close should be consistent with the same liquidation incentive used to derive repayable debt.

For the shares branch, cap by the collateral equivalent of `debt * lif`, not just plain `debt`.

The same consistency fix should be applied to the grace-period and current-period collateral-input branches as well, since they use the same pattern.

As a defensive invariant, if the caller provides enough collateral input to cover the remaining debt under the configured liquidation incentive and the borrower has sufficient collateral, the liquidation should be able to fully close that bucket in the same call.

Developer Response

Fixed in [commit 8d21e95](#).

3.6 Medium Risk Findings

3.6.1 [M-1] In the `IsleV2Service.sol`, the `makeRepay()` under `RepayMode.General` mode allows `shares > 0 && assets > 0`, which can lead to inaccurate `previewRepay()`.

Summary

In the **IsleV2Service** contract, the `makeRepay()` function in **Repay-Mode.General** mode allows `shares > 0 && assets > 0`, which can result in inaccurate repayment amounts.

```
1 // IsleV2Service.sol:648
2 if (mode == RepayMode.General) {
3     require(
4         request.shares != 0 || request.assets != 0,
5         ErrorsLib.IsleV2Service_InvalidParameters(request.
6             shares, request.assets)
7     );
8 }
```

Technical Details

Consider the following scenario:

1. Construct the following request:

```
1 RepayRequest memory exploitRequest = RepayRequest({
2     collateral: address(collateralToken),
3     loan: address(loanToken),
4     onBehalf: SIGNER.addr,
5     shares: borrowSharesBefore / 2,
6     assets: 1000 * (10 ** loanToken.decimals()),
7     nonce: serviceProxy.nonces(SIGNER.addr),
8     deadline: block.timestamp + 1 hours
9 });
```

2. Enter the `_executeRepay()` General branch.
`makeRepay()` is called with `mode = RepayMode.General`:

```
1 require(
2     request.shares != 0 || request.assets != 0,
3     ErrorsLib.IsleV2Service_InvalidParameters(request.
4         shares, request.assets)
5 );
```

`shares != 0` and `assets != 0` evaluates to **true**.

3. Call `market.previewRepay()`.

In General mode:

```
1 transferAmount = market.previewRepay(request.shares,
2     request.assets);
```

4. `_executeRepay()` transfers `transferAmount` from the user and calls the underlying `market.repay()` function:

- Repay debt based on `shares` type.

- Then use the remaining `assets` to repay asset-type debt.

3.6.1.1 PoC Please add the following test to `makeRepay.t.sol` and run `forge test --match-test "test_MakeRepay_BothSharesAndAssets" -vv`

```
1  /// @notice VULNERABILITY: Setting both shares > 0 AND
    assets > 0 causes incorrect preview amount
2  /// @dev This test demonstrates the issue when both
    parameters are set
3  /// Compare this with
    test_MakeRepay_HappyPath_AssetsMode (CORRECT)
4  function test_MakeRepay_BothSharesAndAssets() external
    {
5      uint256 shares = 1000;
6      uint256 assets = 1000 * (10 ** loanToken.decimals
        ());
7
8      // Calculate preview amounts
9      uint256 previewBoth = protocolProxy.previewRepay(
        shares, assets);
10     uint256 previewSharesOnly = protocolProxy.
        previewRepay(shares, 0);
11     uint256 previewAssetsOnly = protocolProxy.
        previewRepay(0, assets);
12
13     console.log("=== VULNERABILITY: Both Parameters
        Set ===");
14     console.log("Shares: ", shares);
15     console.log("Assets: ", assets);
16     console.log("Preview (shares + assets):",
        previewBoth);
17     console.log("Preview (shares only): ",
        previewSharesOnly);
18     console.log("Preview (assets only): ",
        previewAssetsOnly);
19     console.log("Sum (separate): ",
        previewSharesOnly + previewAssetsOnly);
20     console.log("Difference: ",
        previewBoth - (previewSharesOnly +
        previewAssetsOnly));
```

```
1      RepayRequest memory request = RepayRequest({
2          collateral: address(collateralToken),
3          loan: address(loanToken),
4          onBehalf: SIGNER.addr,
5          shares: shares,
6          assets: assets,
7          nonce: 0,
8          deadline: block.timestamp + 1 hours
9      });
10
11      (uint8 v, bytes32 r, bytes32 s) = _signRepay(
12          request, SIGNER.key);
13
14      uint256 signerBalanceBefore = loanToken.balanceOf(
15          SIGNER.addr);
16
17      // Execute - BEFORE FIX: succeeds, AFTER FIX:
18      // reverts
19      vm.prank(OPERATOR.addr);
20      (uint256 returnedShares, uint256 gracePeriodAssets
21          , uint256 currentPeriodAssets) =
22          serviceProxy.makeRepay(request, v, r, s);
23
24      uint256 signerBalanceAfter = loanToken.balanceOf(
25          SIGNER.addr);
26      uint256 actualPaid = signerBalanceBefore -
27          signerBalanceAfter;
28
29      console.log("\n=== EXECUTION RESULT ===");
30      console.log("Previewed amount:      ",
31          previewBoth);
32      console.log("Actually paid:          ",
33          actualPaid);
34      console.log("Returned shares:      ",
35          returnedShares);
36      console.log("Grace period assets:    ",
37          gracePeriodAssets);
38      console.log("Current period assets:    ",
39          currentPeriodAssets);
40      console.log("Difference (preview - actual):",
41          previewBoth > actualPaid ? previewBoth -
42          actualPaid : 0);
43  }
```

Impact

If the `shares` amount already covers most of the debt, the `assets` may not be fully utilized in practice, resulting in `previewRepay()` calculating a transfer amount greater than the actual amount used.

Recommendation

```
1  if (mode == RepayMode.General) {  
2    require(  
3      (request.shares != 0 && request.assets == 0) ||  
4      (request.shares == 0 && request.assets != 0),  
5      ErrorsLib.IsleV2Service_InvalidParameters(request.shares,  
6        request.assets)  
7    );  
8  }
```

Developer Response

Fixed in [commit a815595](#).

3.6.2 [M-2] Grace Period Debt Overwrite in `_accrueCurrentPeriod` Case B if `gracePeriodDuration > periodLength`

Summary

In `MarketShared._accrueCurrentPeriod`, when a borrower's current period expires and transitions to the grace period (Case B), the function uses direct assignment (`=`) instead of addition (`+=`) for `gracePeriodDebtAssets`. If the borrower already has unexpired grace period debt from a prior period (meaning `gracePeriodDuration > periodLength`), it is silently overwritten and becomes unrecoverable — resulting in permanent debt loss and accounting mismatch.

Technical Details

When `_accruePositionView` is called, it processes debt transitions in order:

1. `_accrueGracePeriod` —settles grace period debt **only if it has expired** (`gracePeriodDebtEndTimestamp <= block.timestamp`)
2. `_accrueCurrentPeriod` —transitions current period debt to grace period (Case B) or to borrow shares (Case A)

In Case B of `_accrueCurrentPeriod` (`MarketShared.sol` lines 534–546), the current period debt is moved to the grace period bucket via direct assignment:

```
1 // Case B: Only current period expired, still within grace
  period
2 else if ($borrower.currentDebtEndTimestamp <= block.
  timestamp) {
3     $data.totalCurrentPeriodDebt -= $borrower.
      currentDebtAssets;
4     $data.totalGracePeriodDebt += $borrower.
      currentDebtAssets;
5
6     $borrower.gracePeriodDebtAssets = $borrower.
      currentDebtAssets; // @audit: overwrites, not
      accumulates
7     $borrower.gracePeriodDebtEndTimestamp = $borrower.
      currentDebtEndTimestamp + gracePeriodTimeLength;
8
9     $borrower.currentDebtAssets = 0;
10    $borrower.currentDebtEndTimestamp = nextEnd;
11 }
```

The = on `gracePeriodDebtAssets` silently discards any pre-existing grace period debt that has not yet expired.

This vulnerability is exploitable when `gracePeriodDuration > periodLength`. In `PeriodOracle`, these two parameters are configured independently via `setGracePeriodDuration` and `appendCurrentPeriodDuration`, with no validation enforcing `gracePeriodDuration <= periodLength`.****

Configuration: `periodLength = P`, `gracePeriodDuration = G`,

where $G > P$.

Step	Time	Action	State
1	T_0	Borrower borrows D_1 in Period N	$\text{currentDebtAssets} = D_1$, $\text{currentDebtEndTimeStamp} = E_N$
2	$T_1 \in (E_N, E_N + G)$	Accrue triggers Case B	$\text{gracePeriodDebtAssets} = D_1$, $\text{gracePeriodDebtEndTimeStamp} = E_N + G$, $\text{currentDebtEndTimeStamp} = E_{N+1}$
3	$T_2 \in (T_1, E_{N+1})$	Borrower borrows D_2 in Period N+1	$\text{currentDebtAssets} = D_2$
4	$T_3 \in (E_{N+1}, E_N + G)$	Accrue triggers again	See below

At Step 4, $T_3 > E_{N+1}$ so current period has expired. But $T_3 < E_N + G$ so: - `_accrueGracePeriod: gracePeriodDebtEndTimeStamp =  $E_N + G$  >  $T_3$` → **condition not met, skipped** — D_1 grace debt remains
- `_accrueCurrentPeriod Case B:  $\text{gracePeriodDebtAssets} = D_2$` → **overwrites D_1**

The condition $E_{N+1} < E_N + G$ simplifies to `periodLength < gracePeriodDuration`, which is not prevented anywhere.

After the overwrite: - Borrower's `gracePeriodDebtAssets =  $D_2$` (lost D_1) - `totalGracePeriodDebt` still includes both D_1 and D_2 - Permanent accounting mismatch: `sum(all users'  $\text{gracePeriodDebtAssets}$ )`

< `totalGracePeriodDebt` - `D1` becomes uncollectible —borrower can never be asked to repay it

Impact

- **Debt forgiveness:** Borrowers can escape repayment of prior grace period debt by borrowing again in the next period and waiting for the overwrite.
- **Permanent accounting mismatch:** `totalGracePeriodDebt` becomes inflated relative to the sum of individual positions, which can cascade into incorrect interest accrual, health checks, and liquidation decisions.
- **Supplier loss:** The lost debt directly reduces the assets available to repay suppliers.

Recommendation

Two complementary mitigations are recommended:

1. Add an invariant check in `_accrueCurrentPeriod` Case B to prevent overwriting active grace debt:

```
1 else if ($borrower.currentDebtEndTimestamp <= block.  
  timestamp) {  
2 +   require($borrower.gracePeriodDebtAssets == 0, "active  
  grace debt exists");  
3  
4   $data.totalCurrentPeriodDebt -= $borrower.  
    currentDebtAssets;  
5   $data.totalGracePeriodDebt += $borrower.  
    currentDebtAssets;  
6  
7   $borrower.gracePeriodDebtAssets = $borrower.  
    currentDebtAssets;  
8   // ...  
9 }
```

2. Enforce the parameter constraint at the oracle level in `PeriodOracle.setGracePeriodDuration`:

```
1 function setGracePeriodDuration(uint256 newDuration)
  external onlyRole(CONFIG_ROLE) {
2 +   require(newDuration <= minPeriodLength, "grace period
    exceeds period length");
3     _gracePeriodDuration = newDuration;
4 }
```

Developer Response

Fixed in [commit e74685e](#).

3.6.3 [M-3] Accrual Writeback Overwrites Fee Shares When Borrower Equals Fee Recipient

Summary

`_accruePositionEffect` loads the fee recipient and the borrower position into two separate memory copies from the same storage slot when `borrower == config.recipient`. After `_accruePositionView` credits fee shares exclusively to the `$recipient` copy, the subsequent writeback of `$borrowerPos` overwrites that slot and silently discards all accrued fee shares. Meanwhile `totalSupplyShares` in `$data` is already incremented, creating ghost shares —shares counted in the global total but owned by no one—that permanently dilute all suppliers.

Technical Details

In `_accruePositionEffect`, when `borrower == $market.config.recipient`, the same storage slot will be stored two independent memory variables:

```
1 Position memory $recipient = $market.userPosition[$market.
  config.recipient];
2 Position memory $borrowerPos = $market.userPosition[
  borrower]; // identical slot
```

`_accruePositionView` then processes them independently: -

`$recipient.supplyShares` is incremented by fee shares inside `_collectFee` (called from `_accrueInterestView`, `_accrueGracePeriod`, `_accrueCurrentPeriod`) - `$borrowerPos` receives borrow-side updates (`borrowShares`, `gracePeriodDebtAssets`, `currentDebtAssets`)

At writeback:

```
1 $market.userPosition[$market.config.recipient] =  
  $recipient; // (1) writes fee shares  
2 $market.userPosition[borrower] =  
  $borrowerPos; // (2) overwrites same slot  
3 $market.data = $data;  
                                     // (3)  
  totalSupplyShares already incremented
```

Impact

The protocol fee recipient never receives the supply shares it earned for that accrual cycle. The shares are computed and globally counted but credited to no one.

Recommendation

Add an identity check in `_accruePositionEffect`.

Developer Response

Fixed in [commit dc74909](#).

3.6.4 [M-4] Fee Rates Used at `executePayment` Differ from Creation Time, Enabling In-Flight Over-Charging

Summary

`createPayment()` stores a `PendingPayment` with `payer`, `payee`, `pool`, `loan`, `amount`, and `executeAfter`, but does **not** snapshot `paymentFeeBps` or `protocolFeeBps`. When `executePayment()` runs, it computes fees using the **current** values in service storage. If an

admin changes fee rates between creation and execution, the payer can be charged significantly more than they expected when signing the payment.

Technical Details

`createPayment()` stores a `PendingPayment` with `payer`, `payee`, `pool`, `loan`, `amount`, and `executeAfter`, but does **not** snapshot `paymentFeeBps` or `protocolFeeBps`.

```
1  /// @notice Pending payment data stored after creation.
2  struct PendingPayment {
3      /// @notice The address of the payer.
4      address payer;
5      /// @notice The address of the payee (recipient).
6      address payee;
7      /// @notice The pool address for the market.
8      address pool;
9      /// @notice The loan token address for the market.
10     address loan;
11     /// @notice The payment amount in loan token units.
12     uint256 amount;
13     /// @notice The timestamp after which the payment can
        be executed.
14     uint256 executeAfter;
15 }
```

```
1      // @effect: store pending payment BEFORE external
        calls (CEI pattern)
2      $.pendingPayments[paymentId] = PendingPayment({
3          payer: order.payer,
4          payee: order.payee,
5          pool: address(pool),
6          loan: order.loan,
7          amount: order.amount,
8          executeAfter: block.timestamp + order.timeLock
9      });
```

When `executePayment()` runs, it computes fees using the **current** values in service storage, not from the payment record.

```
1 // @cache: calculate fees
2 uint256 paymentFee = Math.mulDiv(payment.amount, $
    .paymentFeeBps, MAX_BPS);
3 uint256 protocolFee = Math.mulDiv(paymentFee, $.
    protocolFeeBps, MAX_BPS);
4 uint256 supplierFee = paymentFee - protocolFee;
5 uint256 payeeAmount = payment.amount - paymentFee;
```

Consider the following scenario:

1. Payer signs `SimplePayment`: amount = 1000 USDC, timelock = 1 day.
At creation time, `paymentFeeBps` = 100 (1%), so expected fee ≈ 10 USDC, payee receives ≈ 990 USDC.
2. Before execution, a `CONFIG_ROLE` holder calls `setFeeBps` (10000, 5000) (100% payment fee, 50% protocol share).
3. `executePayment` runs: `paymentFee` = $1000 * 10000 / 10000 = 1000$, `payeeAmount` = 0.
4. The payer is charged the full amount as fee; the payee receives nothing.
The payer never agreed to this fee structure.

Impact - Trust assumption: The payer must trust that `CONFIG_ROLE` will not change fees before execution. A malicious or compromised admin can exploit this. - **Payer over-charged:** Fees can be raised between creation and execution, increasing the payer's cost beyond what they expected when signing.

Recommendation

Snapshot fee rates when creating the payment and use them at execution.

Developer Response

Fixed in [commit b584aaf](#).

3.6.5 [M-5] Missing Slippage Protection in Supply and Withdraw Operations

Summary

The `supply` and `withdraw` functions in `SupplierFacet` do not expose slippage parameters. The share-to-asset exchange rate depends on `totalSupplyAssets` and `totalSupplyShares`, which can change between transaction submission and execution. Users may receive fewer shares or assets than expected, and there is no way to enforce a minimum output.

Technical Details

The conversion rate is computed at execution time without any minimum-output checks:

```
1
2  /* ***** ***** ***** ***** ***** ***** */
3  /* Core Implementation
4  /* ***** ***** ***** ***** ***** ***** */
5
6  /// @notice Supplies assets to the market in exchange
    for shares.
7  /// @param assets The amount of assets to supply (0 to
    use shares).
8  /// @param shares The number of shares to mint (0 to
    use assets).
9  /// @param onBehalf The address that will receive the
    supply shares.
10 /// @return The assets supplied and shares minted.
11 function supply(uint256 assets, uint256 shares,
    address onBehalf)
12     external
13     whenNotPaused
14     returns (uint256, uint256)
15 {
16     // Accrue interest first
17     _accruePositionEffect(onBehalf);
18
19     // Execute supply logic
20     (uint256 suppliedAssets, uint256 suppliedShares) =
        _supply(assets, shares, onBehalf);
21
22     // Pull tokens from sender
23     MarketStorage storage $market = __getMarketStorage
        ();
24     IERC20($market.config.loanToken).safeTransferFrom(
        msg.sender, address(this), suppliedAssets);
25
26     return (suppliedAssets, suppliedShares);
27 }
```

```
1    /// @notice Withdraws assets from the market by
    burning shares.
2    /// @param assets The amount of assets to withdraw (0
    to use shares).
3    /// @param shares The number of shares to burn (0 to
    use assets).
4    /// @param onBehalf The address whose shares to burn.
5    /// @param receiver The address that receives the
    withdrawn assets.
6    /// @return The assets withdrawn and shares burned.
7    function withdraw(
8        uint256 assets,
9        uint256 shares,
10       address onBehalf,
11       address receiver
12   )
13   external
14   whenNotPaused
15   onlyRole(SERVICE_ROLE)
16   returns (uint256, uint256)
17   {
18       require(_isSenderAuthorized(onBehalf), ErrorsLib.
           IsleV2Authorization_NotAuthorized());
19       require(receiver != address(0), ErrorsLib.
           IsleV2Common_ZeroAddress());
20
21       // Accrue interest first
22       _accruePositionEffect(onBehalf);
23
24       // Execute withdraw logic
25       (uint256 withdrawnAssets, uint256 withdrawnShares)
           = _withdraw(assets, shares, onBehalf);
26
27       // Push tokens to receiver
28       MarketStorage storage $market = __getMarketStorage
           ();
29       IERC20($market.config.loanToken).safeTransfer(
           receiver, withdrawnAssets);
30
31       return (withdrawnAssets, withdrawnShares);
32   }
```

The rate can change due to: - Interest accrual (e.g. accrueInterest / accruePo-

sition front-run) - changes of `totalSupplyAssets`

ERC-4626 and many lending protocols use `minShares` / `minAssets` parameters for slippage protection; this implementation does not.

Impact - Supply (assets): User supplies X assets but receives fewer shares than expected if the rate moves against them. - Supply (shares): User mints X shares but pays more assets than expected. - Withdraw (assets): User withdraws X assets but burns more shares than expected. - Withdraw (shares): User redeems X shares but receives fewer assets than expected.

Recommendation

Add optional slippage parameters.

Developer Response

Fixed in [commit ac69da7](#).

3.6.6 [M-6] Service previews do not faithfully match execution and can underfund after accrual

Summary

`IsleV2Service` uses `previewSupply()` and `previewRepay()` to decide how many tokens to pull from the user before calling the market. Those preview functions are not execution-faithful: they read the market's non-accrued totals, while the state-changing market entrypoints accrue first and then compute the actual amount consumed. As a result, share-target supply and share-based repay requests can be deterministically underfunded and revert even when the signature is still valid.

Technical Details

In `makeSupply()`:

- the service calls `market.previewSupply(request.assets, request.shares)`;

- it pulls `suppliedAssets` from the user and approves that amount to the market;
- it then calls `market.supply(request.assets, request.shares, request.onBehalf)`.

However, `SupplierFacet.supply()` first calls `_accruePositionEffect(onBehalf)` before executing `_supply()`.

For share-target supply (`assets == 0, shares > 0`), `_supply()` computes:

- `assets = shares.toAssetUp(totalSupplyAssets, totalSupplyShares)`

If interest accrual runs first, `totalSupplyAssets` increases. That makes the post-accrual asset requirement larger than the stale preview used by the service. The service therefore approves and funds too little, and the market's `safeTransferFrom()` reverts.

The same issue exists in `_executeRepay()` for any mode that uses `previewRepay()`:

- general repay when `shares > 0`;
- `makeRepayForShares()`.

`previewRepay()` computes the required loan assets from the current non-accrued `totalBorrowAssets / totalBorrowShares`, but `BorrowerFacet.repay()` and `repayForShares()` call `_accruePositionEffect(onBehalf)` first. If borrow interest accrues, the actual cost of repaying the requested shares increases beyond the amount the service already pulled from the user.

There is an additional nuance on the repay side.

General repay in Isle intentionally allows `shares > 0` and `assets > 0` together. That is not itself a bug: the market repay path first consumes share-based debt and then consumes grace/current-period asset debt.

`previewRepay()` mirrors that intended composite operation by returning:

- the assets needed to repay the requested borrow shares; plus
- the requested direct asset repayment amount.

But execution can still diverge from preview for a second reason: the market caps repayment by the borrower's actual bucket balances at execution time. So a previewed amount can be larger than the actual amount used if the request overstates the repayable shares or asset buckets. The service refunds that excess after the call.

Impact

Valid signed service requests can become unexecutable even without adverse operator behavior. The failure affects the meta-transaction path for share-target supplies and share-based repayments, creating avoidable denial of service and degraded integrator reliability.

On the repay side, the mismatch can also lead to temporary over-pull followed by refund when preview exceeds what the borrower's live bucket balances can actually consume. That secondary effect reinforces that the preview is not execution-faithful, but the security-relevant consequence is the underfund-and-revert path.

Recommendation

Align service funding with the same accrued state and repayment semantics used by market execution.

- add accrued preview functions and use those in the service;
- or have the market expose the actual required amount before transfer;
- or over-pull with an explicit signed max bound and refund the excess after execution.

The key invariant is that the service must never decide the funding amount from a preview that differs materially from live execution. In particular, it

must not rely on stale pre-accrual totals when the actual market call accrues first.

Developer Response

Fixed in [commit 14cefaf](#).

3.6.7 [M-7] topUpFromService mints protocol fee shares against stale supply state

Summary

`topUpFromService()` updates supply assets and mints protocol-fee supply shares without first accruing market interest. If borrow interest has accumulated since the last update, the function uses a stale `totalSupplyAssets` / `totalSupplyShares` exchange rate and can mint too many protocol-fee shares for the same cash amount.

Technical Details

`MarketFacet.topUpFromService()` performs the following logic:

```
1 $market.data.totalSupplyAssets += supplierFee;
2
3 uint256 protocolFeeShares =
4     protocolFee.toShareDown($market.data.totalSupplyAssets
5                             , $market.data.totalSupplyShares);
6 $market.data.totalSupplyAssets += protocolFee;
7 $market.data.totalSupplyShares += protocolFeeShares;
```

The function does not call `_accrueInterestEffect()` first.

That matters because normal market accrual increases supply assets before any share minting based on interest-sensitive state:

```
1 (rate, interest) = _calculateInterest($data.
2     totalBorrowAssets, timeElapsed);
3 $data.totalBorrowAssets += interest;
4 $data.totalSupplyAssets += interest;
5 feeShares = _collectFee(interest, $data, $recipient);
```

So when time has elapsed since `lastUpdate`, `topUpFromService()` reads a stale `totalSupplyAssets` value that is too low relative to the true post-accrual state.

For a fixed `protocolFee`, a lower asset base implies a lower assets-per-share exchange rate, which means:

- `protocolFee.toShareDown(...)` returns more shares than it should;
- `totalSupplyShares` is over-incremented;
- the resulting dilution is larger than intended.

The issue is independent from the known bug where these newly minted shares are not assigned to the configured fee recipient. Even if that separate bug were fixed, `topUpFromService()` would still overmint protocol-fee shares whenever market interest is pending.

Impact

Payment-service fee top-ups can distort share accounting whenever they occur after idle time but before the next market accrual.

This causes excess dilution relative to the cash actually being added and makes the fee-mint outcome depend on whether unrelated accrual happened earlier in the block or transaction sequence.

Recommendation

Accrue the market before using supply totals to mint protocol-fee shares.

- call `_accrueInterestEffect()` at the start of `topUpFromService()`;
- then compute `protocolFeeShares` from the freshly accrued supply totals.

The invariant should be: any share minting based on market-wide supply totals must use the same accrued state that all other supply-share math uses.

Developer Response

Fixed in [commit 5017667](#).

3.6.8 [M-8] Bad debt skipped when collateral exhausted later

Summary

The liquidation flow processes `borrowShares`, `gracePeriodDebtAssets`, and `currentDebtAssets` in three separate stages. Each stage only socializes bad debt for its own bucket if `borrower.collaterals == 0` at that point. If collateral is exhausted in a later stage, any debt remaining from an earlier stage is left outstanding against zero collateral and is not written off during that liquidation.

This does not permanently brick cleanup: a later liquidation can socialize the leftover debt correctly. The bug is that the first liquidation can finalize in a falsely solvent intermediate state, leaving zero-collateral debt on the books until some later actor voluntarily triggers cleanup.

The worst impact is economic misaccounting during that window. Because the leftover `borrowShares` remain in market totals as if they were still collectible, suppliers can continue to accrue value against debt that is already economically unsecured. If no one promptly performs the cleanup liquidation, `totalBorrowAssets` and `totalSupplyAssets` stay overstated, additional interest can continue accruing on the unbacked debt, early exiters can redeem at an inflated price, and the eventual loss is pushed onto whichever suppliers remain when the bad debt is finally recognized.

Technical Details

`__liquidateView()` executes liquidation in this fixed order:

1. `__liquidateForShares()`
2. `__liquidateForGracePeriod()`
3. `__liquidateForCurrentPeriod()`

Each subroutine updates the shared in-memory borrower struct, including `collaterals`, but each bad-debt handler only clears the debt bucket owned by that subroutine.

In particular:

- `__liquidateForShares()` writes off only remaining `borrowShares` when collateral is zero.
- `__liquidateForGracePeriod()` writes off only remaining `gracePeriodDebtAssets` when collateral is zero.
- `__liquidateForCurrentPeriod()` writes off only remaining `currentDebtAssets` when collateral is zero.

There is no final reconciliation step after all three stages complete.

This allows the following reachable state transition:

1. The borrower starts with nonzero `borrowShares`, nonzero grace and/or current debt, and enough collateral to survive the shares stage.
2. The liquidator calls `liquidate()` with nonzero `shares` and nonzero `assets` while `collaterals == 0` input, which is permitted by the input checks.
3. `__liquidateForShares()` repays only part of the borrow-share bucket and leaves some `borrowShares` outstanding because the input `shares` amount was small.
4. `__liquidateForGracePeriod()` or `__liquidateForCurrentPeriod()` uses the separate `assets` input to seize the rest of the borrower's collateral.
5. That later stage detects `borrower.collaterals == 0` and socializes only its own remaining debt bucket.
6. The earlier leftover `borrowShares` remain on the borrower even though the borrower now has zero collateral.

After the write-back, the borrower can end the liquidation with:

- `collaterals == 0`
- `borrowShares > 0`

while market totals still include that borrow-share debt as collectible.

This is a consequence of scoping bad-debt recognition to each subroutine instead of re-evaluating all remaining debt once collateral reaches zero.

Importantly, the leftover debt is not permanently stuck. A second liquidation call against the same borrower will enter `__liquidateForShares()` with `collaterals == 0`, at which point the remaining `borrowShares` are socialized correctly. The issue is therefore not “cleanup is impossible,” but “cleanup is delayed and optional.”

Impact

The protocol can complete a liquidation while leaving uncollateralized debt on the books. That overstates market solvency by keeping `totalBorrowAssets` and `totalSupplyAssets` higher than the assets realistically supporting the pool.

This mispricing can advantage users who exit before cleanup, since share accounting continues to treat the unresolved debt as recoverable. A later liquidation call can clean up the leftover debt, but until that happens losses are recognized too late and can be shifted onto remaining lenders.

The strongest realistic impact is therefore:

- the first liquidation can end in a false-solvency state,
- loss recognition is deferred until a later liquidation,
- and the protocol relies on optional future cleanup rather than enforcing correct bad-debt settlement in the same transaction.

This is why the issue is High rather than Critical. The accounting error is serious and affects a core solvency path, but the state is later-correctable by a subsequent liquidation and is not an immediate one-transaction drain primitive.

Recommendation

Add a final post-processing step after `__liquidateView()` finishes, or refactor the three liquidation subroutines to share a unified bad-debt settlement path.

The key invariant should be:

- if `borrower.collaterals == 0` at the end of liquidation, all remaining debt buckets for that borrower must be written off in the same transaction.

At minimum, after the three stages complete, check whether collateral is zero and then socialize any remaining:

- `borrowShares`
- `gracePeriodDebtAssets`
- `currentDebtAssets`

before persisting borrower and market state.

The protocol should not rely on a second liquidation to restore correct accounting. If the liquidation that exhausts collateral leaves any debt bucket nonzero, that debt should be socialized immediately in the same transaction.

3.6.8.1 Appendix: PoC Add the following code to `test/integration/concrete/liquidation/liquidate/badDebt.t.sol` and run `forge test --match -test test_PoC_BadDebtSkippedWhenCollateralExhaustedLater -vv`:

```
1  import { PAUSE_ROLE, SERVICE_ROLE, LIQUIDATOR_ROLE,  
    CONFIG_ROLE, RISK_ROLE } from "../../../../../src/  
    libraries/ConstantsLib.sol";  
2  
3  ...  
4  
5      /// @notice PoC: a later liquidation stage can exhaust  
        collateral and leave earlier borrow-share debt  
        untouched  
6      /// @dev The first liquidation ends with zero  
        collateral but non-zero borrowShares because only  
        later buckets are socialized.  
7      function  
        test_PoC_BadDebtSkippedWhenCollateralExhaustedLater  
        () external {  
8          uint256 collateralAmt = 100e18; // Worth 100e6 at  
            the initial 1:1 price  
9          uint256 firstBorrow = 15e6; // Will become borrow  
            shares after overdue accrual  
10         uint256 secondBorrow = 15e6; // Will become grace  
            period debt  
11         uint256 thirdBorrow = 10e6; // Will remain current  
            period debt  
12  
13         vm.startPrank(SUPPLIER.addr);  
14         loanToken.approve(address(protocolProxy),  
            SUPPLY_AMOUNT);  
15         protocolProxy.supply(SUPPLY_AMOUNT, 0, SUPPLIER.  
            addr);  
16         vm.stopPrank();  
17  
18         _mintCollateralTokens(BORROWER.addr, collateralAmt  
            );  
19         vm.startPrank(BORROWER.addr);  
20         collateralToken.approve(address(protocolProxy),  
            collateralAmt);  
21         protocolProxy.addCollateral(collateralAmt,  
            BORROWER.addr);  
22         protocolProxy.setAuthorization(SERVICE.addr, true)  
            ;  
23         vm.stopPrank();  
24  
25         vm.prank(SERVICE.addr);  
26         protocolProxy.borrow(firstBorrow, BORROWER.addr,  
            BORROWER.addr);  
27  
28         Position memory posAfterFirstBorrow =  
            protocolProxy.userData(BORROWER.addr);  
29         vm.warp(uint256(posAfterFirstBorrow.  
            currentDebtEndTimestamp) + 7 days + 1);  
30         protocolProxy.accruePosition(BORROWER.addr);  
31
```

```
1 Expected interpretation of the PoC:
2
3 - the first liquidation demonstrates the bug by ending
  with `collaterals == 0` and `borrowShares > 0`
4 - the second liquidation demonstrates that cleanup is
  possible later
5 - therefore the finding is about delayed bad-debt
  recognition and optional future cleanup, not permanent
  unreachability of the bad-debt path
```

Developer Response

Fixed in [commit 3c7d78b](#).

3.6.9 [M-9] Fee configuration changes retroactively redirect pending accrued fees

Summary

`setFeeBps()` and `setFeeRecipient()` update fee configuration immediately, but they do not accrue the market first. Because protocol fee shares are minted only when interest is eventually accrued, changing fee parameters before accrual causes the newly configured fee rate and recipient to apply retroactively to interest that was already earned over the previous interval.

Technical Details

Protocol fee minting is performed inside `_collectFee()` during accrual, not continuously:

```
1 if (interest != 0 && $market.config.feeBps != 0 && $market
   .config.recipient != address(0)) {
2     uint256 feeAssets = interest.mulDiv($market.config.
       feeBps, MAX_BPS);
3     feeShares = feeAssets.toShareDown($data.
       totalSupplyAssets - feeAssets, $data.
       totalSupplyShares);
4
5     $recipient.supplyShares += feeShares;
6     $data.totalSupplyShares += feeShares;
7 }
```

The `interest` used above comes from `_accrueInterestView()`, which accrues over the full interval since `lastUpdate`:

```
1 uint256 timeElapsed = block.timestamp - $market.data.
   lastUpdate;
```

However, the fee configuration setters mutate storage directly without first settling pending accrual:

```
1 function setFeeBps(uint16 newFeeBps) external onlyRole(
   FEE_ROLE) {
2     _configSetFeeBps(newFeeBps);
3 }
4
5 function setFeeRecipient(address newRecipient) external
   onlyRole(FEE_ROLE) {
6     _configSetFeeRecipient(newRecipient);
7 }
```

which flow into:

```
1 function _setFeeBps(uint16 newFeeBps) internal {
2     __getMarketStorage().config.feeBps = newFeeBps;
3 }
4
5 function _setFeeRecipient(address newRecipient) internal {
6     __getMarketStorage().config.recipient = newRecipient;
7 }
```

This produces retroactive fee-accounting errors:

1. `setFeeBps()` before accrual applies the *new* fee percentage to the entire pending interest interval rather than only future accrual;
2. `setFeeRecipient()` before accrual sends the fee shares for the entire pending interval to the *new* recipient rather than the recipient that was configured while the interest was being earned;
3. changing from zero fee or zero recipient to non-zero immediately before accrual captures fees for a period that should have accrued with no protocol fee.

This is the same lazy-accrual hazard as IRM updates, but on the fee-capture side.

Impact

Pending interest can be monetized under the wrong fee schedule and credited to the wrong recipient.

Consequences include:

- supplier dilution beyond the fee policy that was actually in force during the elapsed interval;
- treasury underpayment or overpayment;
- governance or operators being able to redirect already-earned but not-yet-minted protocol revenue.

The issue is privileged, but it still breaks accounting correctness: fee settings for future accrual are being applied to historical accrual.

Recommendation

Accrue the market before changing fee configuration.

- call `_accrueInterestEffect()` before `setFeeBps()` updates `config.feeBps`;
- call `_accrueInterestEffect()` before `setFeeRecipient()` updates `config.recipient`.

The invariant should be: all elapsed interest up to the config change is settled using the old fee rate and old recipient, and only future accrual uses the new fee configuration.

Developer Response

Fixed in [commit b845174](#).

3.6.10 [M-10] Deleverage close factor is not capped at maxCF above main threshold

Summary

`LiquidatorShared.__calcCloseFactor()` linearly interpolates the deleverage close factor from the borrower's LTV, but it never clamps the result to `[minCF, maxCF]`. If a borrower is already above `mainThreshold`, the interpolation quotient exceeds `1e18`, so the returned close factor exceeds the configured `maxCF`.

Technical Details

`deleverage()` accrues the position, checks `_isDeleverageable(borrower)`, and then computes:

- `availableRepayAssets = _positionBorrowedView(borrower).wMulDown(__calcCloseFactor(borrower))`

The close factor helper is:

```
1 uint256 borrowedAssets = _positionBorrowedView(borrower);
2 uint256 collateralQuotedAssets = _convertToLoan(
    $borrowerPos.collaterals);
3 uint256 ltv = borrowedAssets.wDivDown(
    collateralQuotedAssets);
4
5 uint256 quotient = (ltv - secondaryThreshold)
6     .wDivDown(mainThreshold - secondaryThreshold);
7
8 return quotient.wMulDown(maxCF - minCF) + minCF;
```

- The configuration only enforces `secondaryThreshold < mainThreshold` and `minCF <= maxCF <= WAD`.
- `deleverage()` only requires the user to be deleverageable, meaning `borrowed > collateralValue * secondaryThreshold`.
- It does not require the user to still be below `mainThreshold`.
- Therefore, a borrower that is already liquidatable can still go through `deleverage()`.
- For any `ltv > mainThreshold`, the quotient exceeds `1e18`, so the returned close factor exceeds `maxCF`.

This breaks the documented intent that `maxCF` is the maximum percentage of debt repayable in a single deleverage.

One part of the broader original claim is overstated: the subtraction `ltv - secondaryThreshold` is not reachable with `ltv < secondaryThreshold` through the public `deleverage()` flow, because `_isDeleverageable()` already requires the position to exceed the secondary-threshold borrow limit.

Impact

For borrowers above `mainThreshold`, `deleverage()` can authorize repayment above the configured `maxCF` cap. That allows a deleverager to repay and seize more than the protocol's configured maximum for a single deleverage operation, so the configured close-factor ceiling stops being meaningful precisely in the highest-risk region.

Recommendation

Make `__calcCloseFactor()` saturating.

- If `ltv <= secondaryThreshold`, return `minCF`.
- If `ltv >= mainThreshold`, return `maxCF`.
- Only interpolate for `secondaryThreshold < ltv < mainThreshold`.

This guarantees the close factor always stays within configured bounds and preserves the intended meaning of `maxCF`.

Developer Response

Fixed in [commit a1be34f](#).

3.7 Low Risk Findings

3.7.1 [L-1] Missing Event Emission on state update functions

Summary

Some state-changing functions updates the critical rate state variable without emitting an event. This prevents off-chain systems from reliably detecting and indexing rate changes, reducing transparency and observability.

Technical Details

Some state-changing functions updates the critical rate state variable without emitting an event. This prevents off-chain systems from reliably detecting and indexing rate changes, reducing transparency and observability.

For example, the `FixedPriceAdapter` contract updates the critical `rate` state variable in `_setRate()` without emitting an event.

```
1     function _setRate(uint256 _rate) internal {
2         rate = _rate;
3     }
```

The `PeriodOracle` contract update `_gracePeriodDuration` but still has no event emitted.

```
1     function setGracePeriodDuration(uint256 newDuration)
2         external onlyRole(CONFIG_ROLE) {
3         _gracePeriodDuration = newDuration;
4     }
```

Impact This reduces transparency and observability, and it's a best practice to emit event whenever needed.

Recommendation

Declare and emit an event when the state variable is updated.

Developer Response

Fixed in branch: [audit/taichi commit 213fdad](#).

3.7.2 [L-2] Salt Collision in `_calcSalt` Prevents Deployment of Reversed Collateral/Loan Markets

Summary

The `_calcSalt` function in `IsleV2Factory` sorts `collateralToken` and `loanToken` before hashing, treating them as an unordered pair. The sorting causes both to share **the same CREATE2 salt**, so only one of these markets can be deployed and the second deployment fails.

Technical Details

Because collateral and loan have distinct roles, markets like (USDC collateral, ETH loan) and (ETH collateral, USDC loan) are different.

As a result, protocol cannot support both “deposit USDC, borrow ETH” and “deposit ETH, borrow USDC” markets. **Code Snippet**

```
1 // Deploy Diamond with deterministic salt
2 bytes32 salt = _calcSalt(collateralToken,
3   loanToken);
4 IsleV2Diamond diamond = new IsleV2Diamond{ salt:
5   salt }(facetInits, $.facets.initializerFacet,
6   initData);
```

```
1    /// @dev Calculates a deterministic salt from two
    addresses.
2    /// @param inputA The first address.
3    /// @param inputB The second address.
4    /// @return salt The calculated salt.
5    function _calcSalt(address inputA, address inputB)
        internal pure returns (bytes32 salt) {
6        (address a, address b) = inputA > inputB ? (inputB
            , inputA) : (inputA, inputB);
7        salt = keccak256(abi.encode(a, b));
8    }
```

Impact Protocol cannot support both “deposit USDC, borrow ETH” and “deposit ETH, borrow USDC” markets.

Recommendation

Remove the sorting and derive the salt from the ordered pair:

```
1 function _calcSalt(address collateralToken, address
    loanToken) internal pure returns (bytes32 salt) {
2     salt = keccak256(abi.encode(collateralToken, loanToken
        ));
3 }
```

This keeps the salt deterministic while ensuring different markets for different (`collateral`, `loan`) orderings.

Developer Response

Fixed in [commits 5425b06,83c44ac](#).

3.7.3 [L-3] Desynchronization Between defaultAdmin and DEFAULT_ADMIN_ROLE & default-admin handover does not revoke legacy privileged roles

Summary

The `defaultAdmin` and `pendingDefaultAdmin` storage variables in `IsleV2Factory` are intended to track the current `DEFAULT_ADMIN_ROLE`

and support a two-step transfer. However, because the contract inherits OpenZeppelin's `AccessControlUpgradeable`, anyone with `DEFAULT_ADMIN_ROLE` can call `grantRole` and `revokeRole` directly. This bypasses the `proposeDefaultAdmin` / `acceptDefaultAdmin` flow and can desynchronize `defaultAdmin` from the actual `AccessControl` state. Additionally, `AccessControl` allows multiple holders of `DEFAULT_ADMIN_ROLE`, while `defaultAdmin` only tracks a single address. Thus, an outgoing admin can retain the ability to reconfigure facets or authorize upgrades even after governance believes control has been transferred.

Technical Details

`defaultAdmin` is used in two main ways: 1. New market initialization –passed to `createMarket()` and then to each new Diamond's `initializer1`:

```
1      // Build initialization calldata
2      bytes memory initData = abi.encodeCall(
3          IIsleV2Initializer.initializer1,
4          (
5              name,
6              version,
7              $.defaultAdmin,
8              collateralToken,
9              loanToken,
10             oracle,
11             irm,
12             periodOracle,
13             mainThreshold,
14             secondaryThreshold,
15             borrowThreshold,
16             maxCloseFactor,
17             minCloseFactor
18         )
19     );
```

2. Two-step transfer –updated only when `acceptDefaultAdmin()` is called.

As a result, `defaultAdmin` may point to an address that no longer has `DEFAULT_ADMIN_ROLE`, or the actual admin may not be reflected in

`defaultAdmin`. Since `createMarket()` uses `$.defaultAdmin` for new markets, if desynchronized, new markets may receive an incorrect or powerless admin.

similarly, admin rotation doesn't revoke other privileged roles from old admin, for example `CONFIG_ROLE`, `UPGRADE_ROLE`. Old admin could even set multiple `DEFAULTADMIN` role before the admin transfer. As a result, an outgoing admin can retain the ability to reconfigure facets or authorize upgrades even after governance believes control has been transferred.

Impact 1. `defaultAdmin` may point to an address that no longer has `DEFAULT_ADMIN_ROLE` 2. new markets may receive an incorrect or powerless admin. 3. An outgoing admin can retain the ability to reconfigure facets or authorize upgrades even after governance believes control has been transferred.

Recommendation

Override `_grantRole` and `_revokeRole` for `DEFAULT_ADMIN_ROLE` so they either revert or delegate to the two-step flow, ensuring `defaultAdmin` is always updated when the admin changes.

Developer Response

Fixed in [commit 6bf452e](#).

3.7.4 [L-4] Diamond Proxy `receive()` Permanently Locks Native ETH Due to Missing Withdrawal Mechanism

Summary

`IsleV2Diamond` declares `receive()external payable { }` which accepts native ETH transfers, but no facet in the protocol implements a native ETH withdrawal function. Since `receive()` does not route through the Diamond's fallback/delegatecall mechanism, any ETH sent directly to the Diamond proxy is permanently locked with no recovery path.

Technical Details

In `IsleV2Diamond.sol`, the contract inherits OpenZeppelin's `Proxy` and adds an explicit `receive()` function:

```
1 // IsleV2Diamond.sol, line 87
2 receive() external payable { }
```

The OpenZeppelin `Proxy` contract routes calls through `fallback()` → `_implementation()` → `delegatecall`. However, `receive()` is a separate Solidity function that handles plain ETH transfers (empty calldata) and **bypasses `fallback()` entirely**. This means ETH sent without calldata is accepted by the Diamond but never delegated to any facet.

The problem is that the entire protocol operates exclusively on ERC20 tokens via `safeTransfer` / `safeTransferFrom`. There is no mechanism to move native ETH out of the Diamond proxy.

Impact

- **Permanent fund loss:** Any native ETH sent to the Diamond is irrecoverable under normal operation.

Recommendation

If ETH acceptance is intentionally needed (e.g., for future WETH unwrap support), add a rescue function in an admin facet.

Developer Response

Fixed in [commit 3bd0659](#).

3.7.5 [L-5] IRM Change Causes Inconsistent Interest Rates Between Global Accrual and Grace/Current Period Debt Conversion

Summary

When the Interest Rate Model (IRM) is changed via `setIrm()` between global interest accrual and borrower-specific position accrual, grace period and current period debt conversions apply the *new* IRM to the *entire* overdue period—including time when the market was using the *old* IRM for `totalBorrowAssets`. This creates inconsistent pricing: the same time interval is effectively charged at different rates depending on which debt bucket (accumulated vs. grace/current) it belongs to.

Technical Details

Global vs. Per-User Accrual Use Different Timing

- **`accrueInterest()`** accrues on `totalBorrowAssets` using the IRM at call time. Each accrual uses the rate in effect at that moment.
- **`accruePosition(borrower)`** Step 1 does the same. Steps 2–3 convert grace/current debt to borrow shares and charge interest for the *entire* overdue period using the IRM at *conversion* time.

Global accrual (`_accrueInterestView`):

```
1 // src/facets/shared/MarketShared.sol:148-156
2 uint256 timeElapsed = block.timestamp - $market.data.
  lastUpdate;
3
4 if (timeElapsed != 0) {
5     (rate, interest) = _calculateInterest($data.
      totalBorrowAssets, timeElapsed);
6     $data.totalBorrowAssets += interest;
7     // ...
8 }
```

Grace period conversion (`_accrueGracePeriod`):

```
1 // src/facets/shared/MarketShared.sol:439-449
2 uint256 timeElapsed = block.timestamp - $borrower.
  gracePeriodDebtEndTimestamp;
3 // ...
4 if (timeElapsed != 0) {
5     (rate, interest) = _calculateInterest($borrower.
      gracePeriodDebtAssets, timeElapsed);
6     borrowAssetsToSettle += interest;
7 }
```

`_calculateInterest` always uses the current IRM from config:

```
1 // src/facets/shared/MarketShared.sol:563-567
2 if ($market.config.irm != address(0)) {
3     rate = Iirm($market.config.irm).compoundBorrowRate(
      address(this), $market.data, timeElapsed);
4     interest = assets.wMulDown(rate);
5 }
```

```
1 T0: IRM1 active. Borrower has gracePeriodDebtAssets = 100
  (grace expired at T0).
2 T1: accrueInterest() runs. totalBorrowAssets accrues (T1-
  T0) at IRM1.
3 T2: setIrm(IRM2).
4 T3: User triggers accruePosition(borrower).
```

- **totalBorrowAssets:** (T0, T1) at IRM1, (T1, T3) at IRM2.
- **Grace debt conversion:** entire (T0, T3) at IRM2.

For the overlap (T0, T1), accumulated debt used IRM1, but grace debt is charged IRM2 for that same period when it is later converted.

Impact

1. **Unfair treatment:** Borrowers whose grace/current debt converts after an IRM change are charged the new rate for the full overdue period, including time when the old rate was in effect.
2. **Rate arbitrage:** If $IRM2 < IRM1$, borrowers can delay interaction to pay less; if $IRM2 > IRM1$, they are overcharged.

Recommendation

When changing IRM, consider running a keeper to `accruePosition` for affected borrowers before the change, or announce a grace period.

Developer Response

Changing the IRM is an emergency-only admin action, not a routine operation. The minor rate inconsistency during such an event is an acceptable trade-off.

3.7.6 [L-6] Inconsistent Role Requirements for `setFeeBps` Between Market and Service

Summary

The protocol uses different roles for fee configuration across contracts: the **Market** (`ConfigurationFacet`) requires `FEE_ROLE` for `setFeeBps` and `setFeeRecipient`, while **IsleV2Service** requires `CONFIG_ROLE` for `setFeeBps`. This violates the principle of least privilege for fee-related operations on the service and creates inconsistent permission modeling that can lead to overly broad grants or operational confusion.

Technical Details

Check the role design from docs/access-control.md)

Role	Description
<code>FEE_ROLE</code>	Fee configuration: fee basis points, fee recipient
<code>CONFIG_ROLE</code>	Market configuration: period oracle, factory settings

In the market, we have `setFeeBps` and `setFeeRecipient` on the market

correctly use `FEE_ROLE`—a dedicated, narrow role for fee-related configuration.

```
1      function setFeeBps(uint16 paymentFee, uint16
2          protocolFee) external onlyRole(CONFIG_ROLE) {
3          require(paymentFee <= MAX_BPS, ErrorsLib.
4              IsleV2Service_ServiceFeeExceedsMax());
5          require(protocolFee <= MAX_BPS, ErrorsLib.
6              IsleV2Service_ProtocolFeeExceedsMax());
7
8          IsleV2ServiceStorage storage $ =
9              __getIsleV2ServiceStorage();
10         uint16 oldPaymentFeeBps = $.paymentFeeBps;
11         uint16 oldProtocolFeeBps = $.protocolFeeBps;
12
13         $.paymentFeeBps = paymentFee;
14         $.protocolFeeBps = protocolFee;
15
16         emit EventsLib.IsleV2ServiceFeeBpsSet(msg.sender,
17             oldPaymentFeeBps, paymentFee, oldProtocolFeeBps
18             , protocolFee);
19     }
```

On `IsleV2Service`, `setFeeBps` uses `CONFIG_ROLE`, which also gates `setFactory(address)`. The service does not define or use `FEE_ROLE` at all.

```
1  /// @inheritdoc IIsleV2Service
2  function setFeeBps(uint16 paymentFee, uint16
    protocolFee) external onlyRole(CONFIG_ROLE) {
3      require(paymentFee <= MAX_BPS, ErrorsLib.
        IsleV2Service_ServiceFeeExceedsMax());
4      require(protocolFee <= MAX_BPS, ErrorsLib.
        IsleV2Service_ProtocolFeeExceedsMax());
5
6      IsleV2ServiceStorage storage $ =
        __getIsleV2ServiceStorage();
7      uint16 oldPaymentFeeBps = $.paymentFeeBps;
8      uint16 oldProtocolFeeBps = $.protocolFeeBps;
9
10     $.paymentFeeBps = paymentFee;
11     $.protocolFeeBps = protocolFee;
12
13     emit EventsLib.IsleV2ServiceFeeBpsSet(msg.sender,
        oldPaymentFeeBps, paymentFee, oldProtocolFeeBps
        , protocolFee);
14 }
```

Impact This causes the inconsistency: an entity that only needs to change the factory address (e.g., after a factory upgrade) must be granted `CONFIG_ROLE`, which also allows changing payment and protocol fees. There is no way to grant “config-only” without fee control on the service.

Recommendation

Align `IsleV2Service` with the market’s fee role design.

Developer Response

Fixed in [commit d9ca9bc](#)

3.7.7 [L-7] Repay Signatures Do Not Bind to Mode: Operator Can Redirect Signature

Summary

All four repay entrypoints (`makeRepay`, `makeRepayForShares`,

`makeRepayForGracePeriod`, `makeRepayForCurrentPeriod`) accept the same `RepayRequest` struct and use the same `_checkSignatureRepay` verification. The EIP-712 signed payload does **not** include the intended repay mode. As a result, an operator holding `OPERATION_ROLE` can submit a valid user signature to a different entrypoint than the user intended, redirecting the repayment to a different debt bucket (shares, grace period, or current period). The user's intended debt-allocation semantics can be overridden by the operator.

Technical Details

All repay functions share the same signature check and differ only in the `RepayMode` passed to `_executeRepay`:

```
1  function makeRepay(...) {
2      _checkSignatureRepay(request, v, r, s);
3      return _executeRepay(request, RepayMode.General);
4  }
5
6  function makeRepayForShares(...) {
7      _checkSignatureRepay(request, v, r, s);
8      return _executeRepay(request, RepayMode.ForShares)
9      ;
10 }
11
12 function makeRepayForGracePeriod(...) {
13     _checkSignatureRepay(request, v, r, s);
14     return _executeRepay(request, RepayMode.
15         ForGracePeriod);
16 }
17
18 function makeRepayForCurrentPeriod(...) {
19     _checkSignatureRepay(request, v, r, s);
20     return _executeRepay(request, RepayMode.
21         ForCurrentPeriod);
22 }
```

The EIP-712 struct hash only encodes `collateral`, `loan`, `onBehalf`, `shares`, `assets`, `nonce`, and `deadline` with no mode or function selector:

```
1 bytes32 constant RepayRequestTypeHash = keccak256(  
2     "RepayRequest(address collateral,address loan,address  
    onBehalf,uint256 shares,uint256 assets,uint256  
    nonce,uint256 deadline)"  
3 );
```

```
1         bytes32 structHash = keccak256(  
2             abi.encode(  
3                 REPAY_REQUEST_TYPEHASH,  
4                 request.collateral,  
5                 request.loan,  
6                 request.onBehalf,  
7                 request.shares,  
8                 request.assets,  
9                 request.nonce,  
10                request.deadline  
11            )  
12        );
```

Impact A user signs a repay request intending a specific allocation (e.g., pay current-period debt to avoid rollover). The operator submits the same signature to `makeRepayForGracePeriod` instead. The nonce is consumed, the user pays the same amount, but the debt is applied to a different bucket than intended.

Recommendation

Include mode in the signed payload.

Developer Response

Fixed in [commit 8942805](#)

3.7.8 [L-8] Liquidate and Deleverage Lack Slippage Protection for Liquidators

Summary

The `liquidate` and `deleverage` functions in `LiquidatorFacet` do not allow liquidators to specify a minimum amount of collateral (`accumulatedSeizeCollaterals`) they must receive. Between transaction submission and execution, oracle prices, borrower state, or competing liquidations can change, causing the liquidator to receive less collateral than expected and potentially incur a loss.

Technical Details

In both `liquidate` and `deleverage`, the liquidator repays `accumulatedRepayAssets` (loan tokens) and receives `accumulatedSeizeCollaterals` (collateral tokens). The exchange rate is determined by:

- Oracle price via `_convertToCollateral` / `_convertToLoan` in `LiquidatorShared.sol`
- Liquidation incentive factor (LIF) from market config
- Borrower position state (debt, collateral, interest accrual)

```
1
2     // Execute liquidation
3     (
4         uint256 accumulatedRepayShares,
5         uint256 accumulatedRepayAssets,
6         uint256 accumulatedSeizeCollaterals,
7         uint256 accumulatedBadDebtShares,
8         uint256 accumulatedBadDebtAssets
9     ) = _liquidate(shares, assets, collaterals,
10                    borrower);
11
12     // Emit event
13     emit EventsLib.Liquidated(
14         msg.sender,
15         borrower,
16         accumulatedRepayShares,
17         accumulatedRepayAssets,
18         accumulatedSeizeCollaterals,
19         accumulatedBadDebtShares,
20         accumulatedBadDebtAssets
21     );
22
23     // Get market storage for token addresses
24     MarketStorage storage $market = __getMarketStorage
25         ();
26
27     // Push seized collaterals to liquidator
28     IERC20($market.config.collateralToken).
29         safeTransfer(msg.sender,
30                     accumulatedSeizeCollaterals);
```

There is no check that `accumulatedSeizeCollaterals` meets a user-specified minimum. The `onLiquidate` callback receives `repayAssets` but not `accumulatedSeizeCollaterals`.

Impact Liquidators may repay debt but receive less collateral than expected, leading to losses.

Recommendation

Add a `minSeizeCollaterals` parameter to both `liquidate` and `deleverage` for best practice.

Developer Response

Acknowledged as a design trade-off. We do not plan to add `minSeizeCollaterals` to the core `liquidate()` / `deleverage()` functions.

Rationale:

- **Liquidators are not locked behind the Service layer.** The `LiquidatorFacet` role gate can be opened to permissionless mode. Liquidation bots can call the core contract directly or deploy their own wrapper contracts with arbitrary slippage checks —unlike `withdraw`, where `onlyRole(SERVICE_ROLE)` forces all users through the Service with no alternative path.
- **The callback already provides implicit slippage protection.** `liquidate()` follows a flash-style flow: collateral is transferred to the caller before `onLiquidate(repayAssets, data)` is invoked, and loan tokens are pulled after. A bot can inspect the received collateral inside the callback and revert the entire transaction if the amount is unsatisfactory.
- Consistent with Morpho Blue. Morpho Blue's core `liquidate()` similarly has no slippage parameters. Liquidators are expected to handle this in periphery contracts (Bundler / Universal Router). Isle V2 core is aligned with this design.
- Oracle-based pricing, not AMM. Collateral seizure is deterministically computed from a trusted oracle price and the liquidation incentive factor (LIF). There is no AMM-style price impact or MEV sandwich vector.

We are adding slippage protection for `supply` / `withdraw` in the Service layer (<https://github.com/Taichi-Audit-Group/Isle-v2-audit/issues/28>) precisely because end users have no other entry point. Liquidators, by contrast, have full autonomy to manage their own risk.

3.8 Informational Findings

3.8.1 [I-1] Missing ‘__AccessControl_init()’ Call in IsleV2Factory Initializer

Summary

`IsleV2Factory.initialize1()` does not call `__AccessControl_init()` before using AccessControl functions, even though the contract inherits `AccessControlUpgradeable`.

Technical Details

`IsleV2Factory.initialize1()` does not call `__AccessControl_init()` before using AccessControl functions, even though the contract inherits `AccessControlUpgradeable`. Other contracts in the codebase (`IsleV2Service`, `InitializerFacet`) correctly call this parent initializer.

Code Snippet

```
1  /// @inheritdoc IIsleV2FactoryInitializer
2  function initialize1(address initDefaultAdmin)
    external initializer {
3      _grantRole(DEFAULT_ADMIN_ROLE, initDefaultAdmin);
4      __getFactoryStorage().defaultAdmin =
        initDefaultAdmin;
5  }
```

In OpenZeppelin 5.x, `__AccessControl_init()` is empty, so the omission does not cause a functional bug. But it is a best-practice violation and inconsistency with other contracts. If OpenZeppelin later adds logic to `__AccessControl_init()`, this contract may not initialize correctly.

Impact If OpenZeppelin later adds logic to `__AccessControl_init()`, this contract may not initialize correctly.

Recommendation

Call `__AccessControl_init()` at the start of `initialize1()` to follow the upgradeable pattern and align with other contracts.

Developer Response

Fixed in [commit b5ac63f](#).

3.8.2 [I-2] Period Selection Uses “Largest Start” Instead of “Longest Remaining Time”

Summary

`loadInitCurrentPeriod` and `getInitCurrentPeriod` select the overlapping period with the largest `start` time, but the documentation states the goal is to maximize the borrower’s time until their first payment deadline. These criteria differ when the period with the larger start has an earlier end. The implementation can assign borrowers a shorter repayment window than intended.

Technical Details

The protocol intends to choose the period type with the largest start time that contains the current timestamp. This maximizes the borrower’s time until their first payment deadline.

To maximize time until the first payment deadline, the correct criterion is longest remaining time (`period.end - currentTimestamp`), not largest `period.start`.

```
1          // Verify the period actually contains the
           current time
2          if (period.start <= currentTimestamp &&
           currentTimestamp < period.end) {
3              // Select the period with the largest
           start time (closest to current time
           )
4              if (period.start >= bestStart) {
5                  bestEnum = userPeriodEnum;
6                  bestStart = period.start;
7                  bestPeriod = period;
8              }
9          }
```

Impact New borrowers can be assigned a period with a shorter time to first payment deadline than another valid overlapping period. This reduces the intended repayment flexibility and may increase the risk of entering grace period or liquidation sooner than the design intended.

Recommendation

Change the selection criterion from largest `period.start` to longest remaining time.

Developer Response

We acknowledge this observation. The two criteria (“largest start” vs “longest remaining time”) diverge only when different period types are configured with different period lengths. Within a single period type, periods are strictly contiguous (`prev.end == next.start`), so at most one period per type contains a given timestamp.

When all period types share the same duration, “largest start” is equivalent to “longest remaining time” —a later start implies a later end, which implies more remaining time.

Whether different period types have different lengths is an admin configuration decision. The admin controls the period type durations and is expected to understand the selection behavior. We consider this an operational con-

cern rather than a contract logic defect, and will document the selection criterion in the NatSpec to ensure admin awareness.

3.8.3 [I-3] Payment Cancellation Repay-to-Pool Ignores Share-Based Debt

Summary

When `cancelPayment` is called with `repayToPool=true`, the service repays the payer's debt by calling `repay(0, payment.amount, payment.payer)`. Because `shares=0` is hardcoded, only grace-period and current-period debt are targeted. Once the payer's debt has rolled into `borrowShares` (overdue accrual), this repay path does not reduce any debt. The market receives nothing, `actualRepaid` is 0, and the full `payment.amount` is refunded to the payer while the debt remains in the pool.

Technical Details

Debt moves through three buckets in order: - `currentDebtAssets` —created on borrow - `gracePeriodDebtAssets` —when current period ends without full repayment - `borrowShares` —when grace period ends (overdue debt)

`__repayForShares` only reduces `borrowShares` when `shares > 0`:

```
1      {
2          MarketStorage storage $market = __getMarketStorage
            ();
3
4          uint256 repayShares = Math.min(shares, $market.
            userPosition[onBehalf].borrowShares);
5
6          if (repayShares > 0) {
7              uint256 repayAssets = repayShares.toAssetUp(
                $market.data.totalBorrowAssets, $market.
                data.totalBorrowShares);
8
9              accumulatedRepayShares += repayShares;
10             accumulatedRepayAssets += repayAssets;
11
12             $market.data.totalBorrowAssets = UtilsLib.
                zeroFloorOrSub($market.data.
                totalBorrowAssets, repayAssets);
13             $market.data.totalBorrowShares = UtilsLib.
                zeroFloorOrSub($market.data.
                totalBorrowShares, repayShares);
```

With `shares=0`, `repayShares = min(0, borrowShares) = 0`, so borrow-share debt is never touched.

```
1      // @interact: based on repayToPool flag, either
      repay to pool or transfer to user
2      if (cancel.repayToPool) {
3          // Mode A: Repay to pool using balance-before/
          after pattern
4          uint256 balanceBefore = IERC20(payment.loan).
              balanceOf(address(this));
5
6          // @interact: approve loan token to pool for
          repayment
7          IERC20(payment.loan).forceApprove(payment.pool
              , payment.amount);
8
9          // @interact: repay to pool (shares=0, assets=
          amount for assets-based repay)
10         IIsleV2(payment.pool).repay(0, payment.amount,
              payment.payer);
11
12         // @cache: calculate actual repaid amount
13         uint256 balanceAfter = IERC20(payment.loan).
              balanceOf(address(this));
14         uint256 actualRepaid = balanceBefore -
              balanceAfter;
15
16         // @interact: return excess to user if any
17         uint256 excess = payment.amount - actualRepaid
              ;
18         if (excess > 0) {
19             IERC20(payment.loan).safeTransfer(payment.
                payer, excess);
20         }
```

Impact

When debt has rolled into borrowShares, cancel with repayToPool=true does not reduce the payer's debt.

Recommendation

Before calling repay, query the payer's debt composition (e.g. via [previewRepay](#) or a dedicated viewer) and compute the correct shares and assets to cover all buckets, including borrowShares.

Developer Response

The reported scenario —debt rolling into `borrowShares` causing `repay(0, amount, ...)` to silently no-op —cannot occur in practice. The `cancelPayment` time window is bounded by the payment timelock (max 2-3 days in operation), and both the current period and grace period durations in core exceed this upper bound. Debt cannot transition into `borrowShares` within the cancellable window.

That said, we removed the `repayToPool` flag and changed `cancelPayment` to a single path: always repay to pool first, refund excess to payer. Reasons:

- Tokens should not leave the protocol on cancel —debt repayment takes priority
- The `repayToPool` flag delegated the repay-or-refund decision to the signer, creating semantic ambiguity
- A single path eliminates the silent full-refund scenario entirely

commits [1fd0a14](#)

3.8.4 [I-4] Interest Continues to Accumulate During Protocol Pause

Summary

Interest accrual is purely time-based and does not account for the protocol's pause state. When the market is paused, borrowers cannot call `repay()`, `borrow()`, or other user-facing functions due to `whenNotPaused`, but interest continues to accumulate over the pause period. Upon unpauses, the next accrual applies interest for the full elapsed time—including the pause duration—resulting in borrowers being charged/liquidated for a period when they had no ability to reduce their debt.

Technical Details

Interest accrual in `_accrueInterestView` uses `block.timestamp` and `lastUpdate` with no `pause` state being considered:

```
1 // src/facets/shared/MarketShared.sol:140-159
2 function _accrueInterestView(
3     MarketData memory $data,
4     Position memory $recipient
5 )
6     internal
7     view
8     returns (uint256 rate, uint256 interest, uint256
9         feeShares)
9 {
10     MarketStorage storage $market = __getMarketStorage();
11     uint256 timeElapsed = block.timestamp - $market.data.
12         lastUpdate;
13
14     if (timeElapsed != 0) {
15         (rate, interest) = _calculateInterest($data.
16             totalBorrowAssets, timeElapsed);
17         $data.totalBorrowAssets += interest;
18         $data.totalSupplyAssets += interest;
19         feeShares = _collectFee(interest, $data,
20             $recipient);
21     }
22
23     $data.lastUpdate = block.timestamp;
24 }
```

`timeElapsed` always spans from `lastUpdate` to `block.timestamp`, regardless of whether the protocol was paused during that interval.

Borrower-facing functions such as `repay()` are protected by `whenNotPaused`, so borrowers cannot repay while paused:

```
1 // src/facets/BorrowerFacet.sol:142-152
2 function repay(
3     uint256 shares,
4     uint256 assets,
5     address onBehalf
6 )
7     external
8     whenNotPaused
9     returns (uint256 repayShares, uint256
10         repayGracePeriodAssets, uint256
11         repayCurrentPeriodAssets)
12 {
13     // Accrue position first
14     _accruePositionEffect(onBehalf);
15     // ...
16 }
```

The problem is that: `accrueInterest()` and `accruePosition()` on `MarketFacet` are permissionless and have no `whenNotPaused` modifier, so interest can be forced to accrue during pause:

```
1 // src/facets/MarketFacet.sol:47-56
2 /// @notice Accrues interest on the market's total borrow.
3 function accrueInterest() external {
4     _accrueInterestEffect();
5 }
6
7 /// @notice Accrues interest and updates a specific
8     borrower's position.
9 function accruePosition(address borrower) external {
10     _accruePositionEffect(borrower);
11 }
```

reference: <https://solodit.cyfrin.io/issues/m-02-risk-of-mass-liquidation-after-poolasset-pause-and-unpause-due-to-borrow-interest-compounding-implementation-code4rena-benddao-benddao-git>

Impact

1. **Unfair borrower treatment:** Borrowers are charged interest for time when they cannot repay or manage their positions.

2. **Increased debt:** Longer pauses lead to higher accrued interest and larger repayments.

Recommendation

Consider storing pause/unpause timestamp to allow skipping borrow interests compounding for the duration of the pause.

Developer Response

Pause/unpause is strictly an emergency mechanism. In any realistic emergency window, the extra accrued interest is negligible. Introducing pause-aware timestamps or selectively unpausing repay/supply would add complexity to the implementation.

We suggest adjusting the severity level to Info.